

Received January 5, 2021, accepted January 14, 2021, date of publication January 20, 2021, date of current version January 28, 2021.

Digital Object Identifier 10.1109/ACCESS.2021.3053163

A Learn-to-Rank Method for Model-Based Regression Test Case Prioritization

YECHAO HUANG^{ID}, TING SHU^{ID}, AND ZUOHUA DING^{ID}, (Member, IEEE)

School of Information Science and Technology, Zhejiang Sci-Tech University, Hangzhou 310018, China

Corresponding author: Ting Shu (shuting@zstu.edu.cn)

This work was supported in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LY17F020033, in part by the National Natural Science Foundation of China under Grant 61101111 and Grant 61572441, and in part by the Fundamental Research Funds of Zhejiang Sci-Tech University under Grant 2019Q039.

ABSTRACT Regression testing plays an indispensable role in software maintenance, which refers to retest the software following modifications to determine whether the changes have introduced new faults. However, regression testing requires massive amounts of effort to achieve a high fault detection rate. To address this issue, the test case prioritization technique is used to improve the fault detection rate by adjusting the execution order of test cases. For model-based regression test case prioritization, existing approaches have been developed using the single aspect of model-related information extracted from the previous executed test cases. In this paper, a novel learn-to-rank technique is proposed to prioritize test cases by combining the multidimensional features of Extended Finite State Machine (EFSM) under test to improve fault detection rate. Specifically, our method utilizes the random forest algorithm to combine multiple existing heuristic prioritization methods. Detailed experiments are conducted to evaluate the proposed method's performance in terms of Average Percentage Fault Detected (APFD). The experimental results show that the mean APFD value of our method reaches 0.884 for five subject EFSMs, which is 33.9% higher than the compared methods.

INDEX TERMS Regression testing, model based testing, EFSM, test case prioritization, learn to rank.

I. INTRODUCTION

The need for efficient software testing methods increases as software is becoming larger and more complex. Regression testing is a crucial and expensive software engineering activity, which refers to retesting the software after it has been modified. The goal of regression testing is to determine whether the modification has introduced new faults. However, it requires a massive effort to achieve a high fault detection rate. Therefore, a great amount of research attention has been focused on improving the fault detection ability of the regression test cases.

The optimization methods of test cases in regression testing can be divided into three categories: minimization, selection, and prioritization [1]. The main purpose of test suite minimization is to remove redundant test cases from the test suite. However, the reduction of test suite may decrease its fault detection capability. Test case selection and test suite minimization are quite similar. The difference is that the former is based on the changes between the previous version and

the current version of the Software Under Test (SUT), while the latter is based on the metrics measured from one single version of SUT [1]. Different from the above techniques, test case prioritization does not decrease the size of test suite.

Test case prioritization aims to reduce regression testing costs by reordering the execution of test cases. It can help to detect potential faults as early as possible when the test process is resource-constrained. Until now, most test case prioritization techniques are based on the information gathered from source code [2], such as statement coverage, function coverage, etc. However, the scale of source code becomes much larger as the complexity of software increases. For instance, the code of Boeing 787's flight software approximately 14 million lines [3]. It undoubtedly brings an enormous challenge to code-based test case prioritization methods. Therefore, Model-Based Test Case Prioritization (MBTCP) is introduced to speed up the testing process. In MBTCP, the model's dynamic information collected from the test suite execution is utilized to rank test cases. Since the model is a high-level description of software behavior, the MBTCP methods may be simpler and more efficient than the code-based methods [4]. Model-based testing

The associate editor coordinating the review of this manuscript and approving it for publication was Xabier Larrucea^{ID}.

has been widely used in various formal description techniques such as Unified Modeling Language (UML), Finite State Machine (FSM), and Extended Finite State Machine (EFSM). For state-based systems, EFSM considers both the data part and control part of software, which is powerful for capturing the behavior and structure of software.

Existing MBTCP methods based on the EFSM fall into two classes: the modifications at model level or source code level. We only consider the source code changes without involving the model in this paper [3]. In recent years, some heuristics have been proposed to improve the fault detection rate of test suite. However, previous research has also reported that no single heuristic method can always be optimal in all testing situations. Accordingly, can we combine these heuristics to construct a novel heuristic suitable for various test scenarios? It has become a question worth exploring.

Learn to Rank (LTR) [5] is a technique that uses the machine learning algorithm to generate a ranking model. It can be employed in information retrieval, natural language processing, and data mining [6]. It is worth noting that some researchers have successfully applied this technique to the software fault location [7]–[9]. They used the LTR technique to combine multiple sets of localization input features (e.g., suspiciousness scores computed by suspiciousness evaluation formulas) for boosting fault localization. Empirical research shows that methods based on LTR significantly outperform those using one single formula regarding fault localization accuracy. The successful application of LTR technique in software fault location inspired our research idea in this paper.

In this paper, we propose a novel MBTCP method (named LTR-TS) for model-based regression testing. LTR-TS utilizes random forest algorithm to train the ranking model for prioritizing test cases, based on six existing heuristics. In particular, it uses the mutation testing technique to inject faults into the EFSM under test and then adopts existing heuristics to construct the training data set for each EFSM. Next, the derived data set is fed to the learning algorithm to train a ranking model. Based on the built ranking model, LTR-TS effectively prioritizes test cases by making full use of the advantage of each considered heuristic. To evaluate the effectiveness and efficiency of the presented method, a detailed experiment study is performed in five popular protocol EFSMs: Monitor [10], INRES [11], Class II [11], OLSR [12], and SCP [13]. The experimental results suggest that our method is more effective than existing methods in terms of fault detection rate.

The main contributions of this paper are as follows.

- 1) We present a framework that employs the learn-to-rank algorithm to combine six existing MBTCP heuristic methods for improving the Average Percentage Fault Detected (APFD) of regression test cases.
- 2) Five popular LTR algorithms are applied to our framework. The experimental results indicate that random forest algorithm has the best performance in APFD.
- 3) The detailed comparative experiments are conducted to evaluate the effectiveness and efficiency of different

MBTCP methods based on five communication protocol EFSMs.

- 4) The transferability of the LTR ranking model is also investigated. Empirical research finds that the trained ranking model of one EFSM could also be used for other different EFSMs.

The rest of this paper is organized in the following sections: Section II introduces some work on the test case prioritization and LTR technique. Section III introduces the problem of test case prioritization, then provides an overview of EFSM and describes several existing MBTCP methods in detail. Section IV gives some basic definitions used in our method and then shows how we employ the LTR technique to combine existing MBTCP methods. Section V describes the design of experiments, and the discussion of experimental results is also given. Section VI lists some threats to validity. Finally, Section VII summarizes our proposed method and suggests the future work.

II. RELATED WORK

A. TEST CASE PRIORITIZATION

At present, most of the test case prioritization methods are code-based. Elbaum *et al.* [2] implemented many statement coverage and function coverage methods for code-based test case prioritization. Yoo *et al.* [14] used cluster analysis method to sort test cases. Marchetto *et al.* [15] proposed a multi-objective method to prioritize test cases. Their method recovers links among software artifacts and then recognizes the fault-prone portions of software artifacts. Lately, Di Nucci *et al.* [16] proposed a Hypervolume-based Genetic Algorithm (HGA) to sort test cases using multiple test coverage criteria. Their empirical study shows that HGA is more cost-effective compare to five existing techniques. Luo *et al.* [17] compared the static test case prioritization method with the dynamic test case prioritization method. Their experimental results on 58 java programs show that the static technology may have unexpectedly good performance in certain specific situations. The number and type of mutants do not significantly affect the measurement for the effectiveness of test case prioritization. Laali *et al.* [18] used online fault detection information to sort test cases. Khatib-syarbini *et al.* [19] applied the Firefly algorithm to sort test cases by using the similarity distance model. The experiment shows that the Firefly algorithm performs better than existing methods in terms of APFD. Jahan *et al.* [20] presented a semi-automatic risk-based test case prioritization technique using the software modification information and the functions invocation relationship. Solanki *et al.* [21] proposed a method based on the Ant Colony Optimization (ACO) algorithm to sort test cases by using the ant colony's natural food selection strategy.

The MBTCP methods have attracted more researchers' attention in the past decade. Kundu *et al.* [22] used the information from UML sequence diagrams to order test cases. They also proposed three different prioritization metrics that use the message weight and edge weight.

Kaur *et al.* [23] converted the UML activity graph to the Control Flow Graph (CFG) to rank test cases. Ouriques *et al.* [24] used the large industrial systems as experimental subjects to further study the existing MBTCP methods. Tahat *et al.* [3] proposed several test case prioritization methods for EFSM. Their experimental study shows that the MBTCP methods may reduce testing time compared to the code-based methods [4]. Sharma and Singh [25] presented a review about using the ACO algorithm to sort model-based test cases. Lately, Mohdshafie *et al.* [26] integrated the selective method and the even spread count-based method to order test cases employing a scrutinized ordering criterion. This integrated algorithm performs better than a single algorithm in terms of APFD.

B. LEARN TO RANK

In information retrieval, learn to rank (LTR) is a technique for training models that rank documents according to their relevance, preference, or importance [5]. Liu [5] analyzed the strengths and weaknesses of different LTR algorithms and studied which data sets are fair for different LTR algorithms. Hu *et al.* [27] proposed a method based on reinforcement learning for ranking the items in e-commerce platforms. The experimental results show that the algorithm increases the trading volume in the *TaoBao* search engine by more than 30%. Niu *et al.* [28] proposed a code example search approach that applies the LTR technique to rank the code examples relevant to user's query. Their approach is more effective than the existing ranking approaches in terms of Normalized Discounted Cumulative Gain (NDCG).

In recent years, the LTR technique has been frequently used in software testing. Ye *et al.* [29] introduced an adaptive method to rank all source files of a project for the probability of introducing new bugs after modification. The ranking score of each source file is computed as a weighted combination of an array of features based on the given bug report. The weights are trained by LTR algorithm based on previous bug reports. Yang *et al.* [30] applied the LTR technique to train a defect prediction model for ranking task. The proposed method is more efficient than the existing algorithms on real-world data sets. Le *et al.* [7] presented a new fault localization method based on the LTR technique to reduce debugging effort. Firstly, the method learns a model that relates software metrics to the number of faults. Then, the software modules with more faults are assigned higher test priority. Kim *et al.* [8] also used the LTR technique to precise fault localization for software testing. Their method uses genetic programming to learn a ranking model based on the software's dynamic and static characteristics. However, the LTR technique has not been applied to MBTCP so far.

III. PRELIMINARIES

A. THE DEFINITION OF TEST CASE PRIORITIZATION

The test case prioritization focuses on the execution order of test cases in regression testing. It assumes that different

test cases contribute adversely to the test. The test case with better fault detection capability should be executed earlier. The benefit of test case prioritization is that the reordered test suite can detect more faults in a limited time. The test case prioritization problem can be formally defined as follows [31]:

Given: $TS, P_{TS}, f : P_{TS} \rightarrow R$.

Problem: Find $TS' \in P_{TS}$ such that $(\forall TS'')(TS'' \in P_{TS})(TS'' \neq TS') [f(TS') \geq f(TS'')]$.

Here, P_{TS} indicates all possible orderings of the test suite TS . The function f yields a reward R for P_{TS} . Higher rewards are preferable to lower ones. TS' represents the optimal permutation of TS , and TS'' represents the other permutations of TS .

Test case prioritization methods can be divided into two categories: the code-based method and the model-based method. So far, most of them are code-based. However, the efficiency of regression testing is limited by the size and complexity of the code under test. On the other hand, model-based testing utilizes information from the abstract model of software. Consequently, it is more efficient than code-based testing in both the execution and prioritization of test cases [26].

B. THE RELEVANT DEFINITIONS OF EFSM

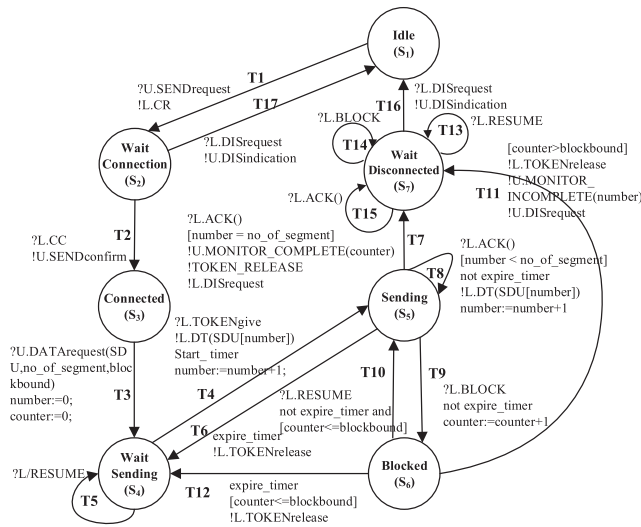
The MBTCP methods discussed in this paper are based on EFSM models. The deterministic EFSM can be described as a 6-tuple $M = \langle S, s_0, V, I, O, T \rangle$ [32], where S is a set of states. $s_0 \in S$, and s_0 is the initial state. $V = (v_1, v_2, \dots, v_n)$ is a set of variables. I is a set of input events, and O signifies a set of output events. T contains all transitions of the model. Each transition can be expressed as a 6-tuple $\tau = \langle s^s, i, p, a, o, s^e \rangle$. $s^s \in S$ and $s^e \in S$ denote the start state and the end state of τ , respectively. $i \in I$ and $o \in O$ represent the input event and the output event, respectively. p denotes the predicate condition of the transition. A transition can be executed when it receives an input event and its p is satisfied. Finally, a denotes the action for variables.

As an example, Fig. 1 shows the EFSM model of monitor network protocol (abbreviated as Monitor). The function of this protocol is to monitor the traffic of network. Monitor contains 17 transitions labeled from $T1$ to $T17$ and 7 states labeled from S_1 to S_7 . Its variable set is $V = (\text{number}, \text{counter}, \text{no_of_segment}, \text{blockbound}, \text{SDU})$. Each variable in V is an integer. *number* and *counter* store the execution times for some transitions. *no_of_segment* and *blockbound* are used as the upper boundary. Finally, *SDU* is an input and output variable.

In transition $T1$, $?U.SENDrequest$ means that the event *SENDrequest* is an input from the interaction point U , and $!L.CR$ means that the event *CR* is an output from the interaction point L . In transition $T3$, *SDU*, *no_of_segment*, and *blockbound* are inputs for Monitor, *number* and *counter* are initialized to 0. In transition $T4$, after Monitor holds token, the timer will start, and *number* will increase by one, then *SDU* and *number* will output. Transition $T7$ is feasible only if the value of *number* is equal to *no_of_segment*.

TABLE 1. The catalog of existing MBTCP methods.

Code	Mnemonic	Description
H1	random	random ordering
H2	optimal	ordered to optimize rate of fault detection
H3	selective	prioritize in order of whether marked transition is included
H4	count-based	prioritize in order of the count of marked transitions
H5	round-robin count-based	prioritize the test case using the round-robin strategy and the count of marked transitions
H6	even spread count-based	prioritize the test case using the even spread strategy and the count of marked transitions
H7	frequency-based	prioritize in order of the frequency of marked transitions
H8	round-robin frequency-based	prioritize the test case using the round-robin strategy and the frequency of marked transitions

**FIGURE 1.** The EFSM specification of network monitor protocol.

After executing $T7$, the token will release, and $counter$ will output. After executing $T9$, $counter$ will increase by one. $T11$ is executable only if $counter$ is larger than $blockbound$. More detailed explanation for each transition in Monitor can be obtained in [10].

The configuration of EFSM is a tuple (s, v) , where s denotes a state and v is a set of variables [33]. The test case t of EFSM can be defined as a sequence of *input/output* pairs that starts from an initial configuration to an end configuration. For example, suppose the initial configuration and end configuration of Monitor are $(S_1, (0, 0, 0, 0, 0))$ and $(S_4, (0, 0, 2, 2, 1))$, then a feasible sequence could be $(S_1, (0, 0, 0, 0, 0)) \xrightarrow{?U.SENDrequest / !L.CR} (S_2, (0, 0, 0, 0, 0)) \xrightarrow{?L.CC / !U.SENDconfirm} (S_3, (0, 0, 0, 0, 0)) \xrightarrow{?U.DATArequest(1, 2, 2) / -} (S_4, (0, 0, 2, 2, 1))$. The corresponding test case can be briefly recorded as $t = \{T1, T2, T3\}$.

C. THE EXISTING MBTCP METHODS

The MBTCP methods discussed in this paper are based on some minor modifications in the software under testing. These modifications will not cause changes on the model specification. In other words, the transitions and states of the model will not increase or decrease, and the starting and ending states of the transitions will not be modified.

In recent years, researchers have paid more attention to such minor modifications [3], [26]. First of all, most of faults that appear in the software are minor bugs, such as calculation errors, wrong arithmetic symbols, and boundary-related errors. These minor bugs are generally difficult to detect. Besides, the model specification is an abstraction of software that hides many implementation details. If the model specification is not changed, it is not feasible to detect these minor bugs by comparing different versions of model specification. In addition, testers can also obtain the information about the data flow path and the dependence between different transitions in EFSM by detecting these minor bugs. On the other hand, when the EFSM implementation is significantly modified (e.g., adding or removing a functional module), the corresponding EFSM specification may need to be redesigned. In this case, the fault detection capability of test suite may be reduced, and testers probably focus more on the selection or generation of test cases rather than prioritization.

For these minor modifications, testers need to mark the corresponding model transitions based on the source code's modifications. Tahat *et al.* [3] named these transitions as **marked transitions**. For example, the transition $T4$ in Monitor will be marked after the action $(number := number + 1)$ is modified to $(number := number - 1)$ in the source code. Because the modified transition has a high risk of introducing new faults and the tester cannot know which transitions will reveal the faults before test suite execution, the test case with marked transitions should be executed earlier. Since the MBTCP method needs to be applied before the test suite execution, the only information it can be used is which transitions are modified.

Table 1 provides a brief description of eight existing MBTCP methods. H1 was utilized as a baseline in our comparative experiment. We used H2 to set training labels and used H3-H8 to set feature values. In addition, H2 was presented by Rothermel *et al.* [31], H3-H8 were proposed by Tahat *et al.* [3]. Next, we will describe more details of these methods.

1) H1: RANDOM PRIORITIZATION

H1 applies the random strategy to sort test cases. It can provide a lower limit of APFD values in our experiment. Since H1 does not use any information about EFSM for ranking,

Algorithm 1 Random Prioritization**Input:** Test Suite: $TS = \{t_1, \dots, t_n\}$ **Output:** Prioritized test sequence: Sq

- 1: Set sequence Sq as empty
- 2: **while** $TS \neq \emptyset$ **do**
- 3: Randomly select the $t \in TS$ and remove it from TS
- 4: Append t to Sq
- 5: **end while**
- 6: **return** Sq

TABLE 2. Example 1.

Test Cases	Faults			
	1	2	3	4
t_1	X	X		
t_2	X			X
t_3		X	X	

the effectiveness of the new intelligent method should be better than H1 in theory. Algorithm 1 is the pseudocode of the H1.

2) H2: OPTIMAL PRIORITIZATION

H2 iteratively selects the test case that reveals the largest number of unrevealed faults. It needs a priori knowledge of which test cases expose which faults. Because the faults are known in our controlled experiment, we can get this information after executing the test suite. Although this method has no practical significance, it can provide an upper bound for APFD values in the experiment. Meanwhile, we used it to replace the manual labeling process in supervised learning. For better understanding, we provide the pseudocode Algorithm 2 for H2. In this algorithm, Max stores the test cases that reveal the most faults.

Algorithm 2 Optimal Prioritization**Input:** Test Suite: $TS = \{t_1, \dots, t_n\}$, $faultlist$ **Output:** Prioritized test sequence: Sq

- 1: Set sequence Sq as empty
- 2: **while** $TS \neq \emptyset$ **do**
- 3: $Max = \emptyset$
- 4: **for** $t \in TS$ **do**
- 5: **if** t that reveals the maximum faults **then**
- 6: $Max = Max \cup \{t\}$
- 7: Remove t from TS
- 8: Remove the faults revealed by t from $faultlist$
- 9: **end if**
- 10: **end for**
- 11: **while** $Max \neq \emptyset$ **do**
- 12: Randomly select the $t \in Max$ and remove it from Max
- 13: Append t to Sq
- 14: **end while**
- 15: **end while**
- 16: **return** Sq

It is worth noting that this method may not get the best ranking result [31]. We provide Example 1 to illustrate this special case. In Table 2, there are four known faults and three test cases. The information about which test cases expose which faults are shown in the table. The optimal ordering should be t_2, t_3, t_1 or t_3, t_2, t_1 , but H2 may generate t_1, t_3, t_2 . Because all test cases can reveal the same number of faults when H2 makes the first selection.

3) H3: SELECTIVE PRIORITIZATION

The test case that contains at least one marked transition is executed earlier in H3. If the test case contains at least one marked transition, it will be grouped into the high priority set TS_H . Otherwise, it will be grouped into the low priority set TS_L . Then, H3 randomly selects the test cases in TS_H . After all test cases in TS_H are selected, the test cases in TS_L will be randomly selected. Algorithm 3 is the pseudocode of H3.

Algorithm 3 Selective Prioritization**Input:** Test Suite: $TS = \{t_1, \dots, t_n\}$, a set of marked transitions: MT **Output:** Prioritized test sequence: Sq

- 1: Set sequence Sq as empty, $TS_H = \emptyset, TS_L = \emptyset$
- 2: **for** $t \in TS$ **do**
- 3: **if** t contains the marked transition in MT **then**
- 4: $TS_H = TS_H \cup \{t\}$
- 5: **else**
- 6: $TS_L = TS_L \cup \{t\}$
- 7: **end if**
- 8: **end for**
- 9: **for** $t \in TS_H$ **do**
- 10: Randomly select $t \in TS_H$ and remove it from TS_H
- 11: Append t to Sq
- 12: **end for**
- 13: **for** $t \in TS_L$ **do**
- 14: Randomly select $t \in TS_L$ and remove it from TS_L
- 15: Append t to Sq
- 16: **end for**
- 17: **return** Sq

In Table 3, we provide Example 2 for further explanation. Assume there are eight test cases $TS = \{t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8\}$ and four marked transitions $MT = \{T4, T8, T9, T11\}$ in Monitor. Table 3 shows the contained marked transitions for each test case. In the same test case, each marked transition can be executed more than once (e.g., $T11$ is feasible only after $T9$ is executed several times). Because t_7 and t_8 do not contain any marked transition in MT , their content is empty in the table (e.g., t_7 could be $\{T1, T17, T2\}$).

In Example 2, we can observe that t_1-t_6 all contain at least once marked transition. These test cases are grouped into TS_H in line 4 of Algorithm 3. Correspondingly, t_7 and t_8 are grouped into TS_L in line 6. Next, H3 randomly selects t_1-t_6 in lines 9-12. After TS_H is empty, H3 will randomly select t_7 and t_8 in lines 13-16. Finally, the following prioritized sequence may be created $Sq = \{t_4, t_2, t_3, t_1, t_6, t_5, t_8, t_7\}$.

TABLE 3. Example 2.

$t_1 : T4$	$t_5 : T4, T9$
$t_2 : T4$	$t_6 : T4, T8, T9, T9, T9, T9, T11$
$t_3 : T4, T8, T8$	$t_7 :$
$t_4 : T4, T8$	$t_8 :$

4) H4: COUNT-BASED PRIORITIZATION

The test case that contains more marked transitions will get a higher priority in H4. Let $A(t)$ be a set of marked transitions in t . $|A(t)|$ is the number of mark transitions in t (e.g., $|A(t_5)| = 2$, $|A(t_6)| = 4$). Let $B(k) \subseteq TS$ ($0 < k < |MT|$) be the subset of TS such that for each test case t in $B(k)$, $k = |A(t)|$. The count of marked transitions of each test case in $B(k)$ is equal to k . Notice that these subsets are disjoint sets. That is, each test case is located only in one subset. If $i > j$, subset $B(i)$ has a higher priority than $B(j)$. Let A_{max} be the maximum value among $|A(t_1)|, \dots, |A(t_n)|$.

H4 divides the test suite into the following subsets with decreasing priority: $B(A_{max}), B(A_{max}-1), \dots, B(1), B(0)$. The test cases in $B(A_{max})$ have the highest priority. Firstly, it randomly selects test cases in $B(A_{max})$. After all test cases in $B(A_{max})$ are selected, it will select test cases in $B(A_{max}-1)$. In the end, all test cases in $B(0)$ are randomly selected. Algorithm 4 is the pseudocode of H4.

Algorithm 4 Count-Based Prioritization

Input: Test Suite: $TS = \{t_1, \dots, t_n\}$, sets of marked transitions for each test: $\{A(t_i)\}_{i=1}^n$, a set of marked transitions: MT

Output: Prioritized test sequence: Sq

```

1: Set sequence  $Sq$  as empty
2: for  $k = 0 \rightarrow A_{max}$  do
3:    $B(k) = \emptyset$ 
4: end for
5: for  $t \in TS$  do
6:    $k = |A(t)|$ ;  $B(k) = B(k) \cup \{t\}$ 
7: end for
8: for  $k = A_{max} \rightarrow 0$  do
9:   while  $B(k) \neq \emptyset$  do
10:    Randomly select  $t \in B(k)$  and remove it from  $B(k)$ 
11:    Append  $t$  to  $Sq$ 
12:   end while
13: end for
14: return  $Sq$ 

```

Consider Example 2 in Table 3, the sets of $A(t)$ are as follows: $A(t_1) = \{T4\}$, $A(t_2) = \{T4\}$, $A(t_3) = \{T4, T8\}$, $A(t_4) = \{T4, T8\}$, $A(t_5) = \{T4, T9\}$, $A(t_6) = \{T4, T8, T9, T11\}$, $A(t_7) = \emptyset$, $A(t_8) = \emptyset$. And the sets of $|A(t)|$ are as follows: $|A(t_1)| = 1$, $|A(t_2)| = 1$, $|A(t_3)| = 2$, $|A(t_4)| = 2$, $|A(t_5)| = 2$, $|A(t_6)| = 4$, $|A(t_7)| = 0$, $|A(t_8)| = 0$. In lines 2-7 of Algorithm 4, the following subsets of TS can be obtained: $B(4) = \{t_6\}$, $B(2) = \{t_3, t_4, t_5\}$, $B(1) = \{t_1, t_2\}$, $B(0) = \{t_7, t_8\}$. In lines 8-13, all test cases in $B(4)$ are randomly selected. After that, all

test cases in $B(2)$ are randomly selected, followed by $B(1)$. Finally, the test cases in $B(0)$ are randomly selected. As a result, the following prioritized sequence could be created $Sq = \{t_6, t_3, t_5, t_4, t_1, t_2, t_8, t_7\}$.

5) H5: ROUND-ROBIN COUNT-BASED PRIORITIZATION

H5 applies the round-robin strategy to H4, allowing low-priority test cases to be selected ahead of time. H5 will loop through each subset in sequence: $B(A_{max}), B(A_{max}-1), \dots, B(1)$ and randomly select one test case when getting into the corresponding subset. After all test cases in above sequence have been selected, all test cases from $B(0)$ will be randomly selected at the end. Algorithm 5 is the pseudocode of H5.

Algorithm 5 Round-Robin Count-Based Prioritization

Input: Test Suite: $TS = \{t_1, \dots, t_n\}$, sets of marked transitions for each test: $\{A(t_i)\}_{i=1}^n$, a set of marked transitions: MT

Output: Prioritized test sequence: Sq

```

1: Set sequence  $Sq$  as empty,  $p = 0$ 
2: for  $k = 0 \rightarrow A_{max}$  do
3:    $B(k) = \emptyset$ 
4: end for
5: for  $t \in TS$  do
6:    $k = |A(t)|$ ;  $B(k) = B(k) \cup \{t\}$ 
7: end for
8: while  $p < |TS| - |B(0)|$  do
9:   for  $k = A_{max} \rightarrow 1$  do
10:    if  $B(k) \neq \emptyset$  then
11:      Randomly select  $t \in B(k)$  and remove it from  $B(k)$ 
12:      Append  $t$  to  $Sq$ 
13:       $p = p + 1$ 
14:    end if
15:   end for
16: end while
17: while  $B(0) \neq \emptyset$  do
18:   Randomly select  $t \in B(0)$  and remove it from  $B(0)$ 
19:   Append  $t$  to  $Sq$ 
20: end while
21: return  $Sq$ 

```

In Example 2, H5 first randomly selects a test case from $B(4)$, then from $B(2)$, and finally from $B(1)$. These steps will repeat until all test cases from $B(4)$ to $B(1)$ are all selected in lines 8-16 of Algorithm 5. In the end, H5 randomly selects all test cases from $B(0)$ in lines 17-20. As a result, the following prioritized sequence might be created $Sq = \{t_6, t_3, t_1, t_4, t_2, t_5, t_8, t_7\}$.

6) H6: EVEN SPREAD COUNT-BASED PRIORITIZATION

The basic idea of H6 is that each marked transition should have the same opportunity to be executed during testing. This method will record the executed count of marked transition in selected test cases. If the selected test case contains a marked transition at least once, the executed count for that marked

transition will increase by one. Those not yet selected test cases with less executed marked transitions will get higher priority to be selected. Similar to other heuristics, the test cases without marked transition are randomly selected at the end.

Algorithm 6 is the pseudocode of H6. In the algorithm, $count(T)$ records the executed count of marked transition T in the selected test cases. Set E stores the marked transitions with the minimum execution count.

Algorithm 6 Even Spread Count-Based Prioritization

Input: Test Suite: $TS = \{t_1, \dots, t_n\}$, sets of marked transitions for each test: $\{A(t_i)\}_{i=1}^n$, a set of marked transitions: MT

Output: Prioritized test sequence: Sq

```

1: for  $T \in MT$  do
2:    $count(T) = 0$ 
3: end for
4: Set sequence  $Sq$  as empty
5: Randomly select  $t \in TS$  for which  $A(t) \neq \emptyset$  and remove it from  $TS$ 
6: Append  $t$  to  $Sq$ 
7: for  $T \in A(t)$  do
8:    $count(T) = count(T) + 1$ 
9: end for
10: while exists  $t \in TS$  that  $A(t) \neq \emptyset$  do
11:    $E = \emptyset$ ;  $min = MaxValue$ 
12:   for  $T \in MT$  do
13:     if  $min == count(T)$  then
14:        $E = E \cup \{T\}$ 
15:     end if
16:     if  $min > count(T)$  then
17:        $min = count(T)$ ;  $E = \{T\}$ 
18:     end if
19:   end for
20:   Randomly select transition  $T_a \in E$ 
21:   Randomly select  $t \in TS$  that  $T_a \in A(t)$ 
22:   if  $t$  is not null then
23:     Remove  $t$  from  $TS$ 
24:     Append  $t$  to  $Sq$ 
25:     for  $T \in A(t)$  do
26:        $count(T) = count(T) + 1$ 
27:     end for
28:   else
29:     Remove  $T_a \in MT$ 
30:   end if
31: end while
32: while  $TS \neq \emptyset$  do
33:   Randomly select  $t \in TS$  and remove it from  $TS$ 
34:   Append  $t$  to  $Sq$ 
35: end while
36: return  $Sq$ 

```

Likewise, we use Example 2 for further explanation. In lines 5-9 of Algorithm 6, if H6 randomly selects t_3 , the set of $count$ will update to: $count(T4) = 1$, $count(T8) = 1$,

$count(T9) = 0$, $count(T11) = 0$. Next, H6 updates E to $\{T9, T11\}$ in lines 12-19. If $T11$ is randomly chosen from E in line 20, H6 will select t_6 in lines 21-24 because only t_6 contains $T11$ in the remaining test cases. Then, the set of $count$ is updated to: $count(T4) = 2$, $count(T8) = 2$, $count(T9) = 1$, $count(T11) = 1$ in lines 25-27. In the next iteration, E is updated to $\{T9, T11\}$. If $T11$ is randomly chosen from E , H6 will remove $T11$ from MT in line 29 because none of the remaining test cases contain $T11$. The above steps (lines 10-31) will repeat until all test cases that contain at least one marked transition are selected. In the end, t_7 and t_8 are randomly selected in lines 32-35. As a result, the following prioritized sequence may be created $Sq = \{t_3, t_6, t_5, t_4, t_2, t_1, t_8, t_7\}$

7) H7: FREQUENCY-BASED PRIORITIZATION

In H7, the test case with a higher frequency of marked transitions will be executed earlier. Let $C(t)$ be a set of mark transitions of test case t . Notice that $C(t)$ is different from $A(t)$ because it may contain repetitions of the same marked transitions. $|C(t)|$ represents the frequency of mark transitions in t (e.g., $|C(t_5)| = 2$, $|C(t_6)| = 7$). Let $B(k) \subseteq TS$ ($0 \leq k \leq C_{max}$) be the subset of TS such that for each test case t in $B(k)$, $k = |C(t)|$. The frequency of marked transitions of each test case in $B(k)$ is equal to k . C_{max} is the maximum value among $|C(t_1)|, \dots, |C(t_n)|$. If $i > j$, the test case in $B(i)$ will have a higher priority than the test case in $B(j)$.

H7 divides TS into the following subsets with decreasing priority: $B(C_{max}), B(C_{max}-1), \dots, B(1), B(0)$. It first randomly selects all test cases in $B(C_{max})$ and then selects all test cases in $B(C_{max}-1)$. All test cases in $B(0)$ will be randomly selected at the end. Algorithm 7 is the pseudocode of H7.

Algorithm 7 Frequency-Based Prioritization

Input: Test Suite: $TS = \{t_1, \dots, t_n\}$, sets of marked transitions for each test: $\{C(t_i)\}_{i=1}^n$, a set of marked transitions: MT

Output: Prioritized test sequence: Sq

```

1: Set sequence  $Sq$  as empty
2: for  $k = 0 \rightarrow C_{max}$  do
3:    $B(k) = \emptyset$ 
4: end for
5: for  $t \in TS$  do
6:    $k = |C(t)|$ ;  $B(k) = B(k) \cup \{t\}$ 
7: end for
8: for  $k = C_{max} \rightarrow 0$  do
9:   while  $B(k) \neq \emptyset$  do
10:    Randomly select  $t \in B(k)$  and remove it from  $B(k)$ 
11:    Append  $t$  to  $Sq$ 
12:   end while
13: end for
14: return  $Sq$ 

```

In Example 2, the sets of $C(t)$ are as follows: $C(t_1) = \{T4\}$, $C(t_2) = \{T4\}$, $C(t_3) = \{T4, T8, T8\}$, $C(t_4) = \{T4, T8\}$, $C(t_5) = \{T4, T9\}$, $C(t_6) = \{T4, T8, T9, T9, T9\}$,

$T9, T11\}$, $C(t_7) = \emptyset$, $C(t_8) = \emptyset$. The sets of $|C(t)|$ are as follows: $|C(t_1)| = 1$, $|C(t_2)| = 1$, $|C(t_3)| = 3$, $|C(t_4)| = 2$, $|C(t_5)| = 2$, $|C(t_6)| = 7$, $|C(t_7)| = 0$, $|C(t_8)| = 0$. In lines 5-7 of Algorithm 7, the following subsets of TS can be obtained: $B(7) = \{t_6\}$, $B(3) = \{t_3\}$, $B(2) = \{t_4, t_5\}$, $B(1) = \{t_1, t_2\}$, $B(0) = \{t_7, t_8\}$. In lines 8-13, all test cases in $B(7)$ are selected, then all test cases in $B(3)$ are randomly selected, followed by $B(2)$ and $B(1)$. Lastly, all test cases in $B(0)$ are randomly selected. As a result, the following prioritized sequence could be created $Sq = \{t_6, t_3, t_5, t_4, t_1, t_2, t_8, t_7\}$.

8) H8: ROUND-ROBIN FREQUENCY-BASED PRIORITIZATION

Based on H7, H8 uses the round-robin strategy to make the test case with low priority can be selected earlier. H8 will loop through the following subsets of TS : $B(C_{max}), B(C_{max}-1), \dots, B(1)$ and randomly select one test case when getting into the corresponding subset. After all test cases in above subsets have been selected, the test cases from $B(0)$ will be randomly selected at the end. Algorithm 8 is the pseudocode of H8.

Algorithm 8 Round-Robin Frequency-Based Prioritization

Input: Test Suite: $TS = \{t_1, \dots, t_n\}$, sets of marked transitions for each test: $\{C(t_i)\}_{i=1}^n$, a set of marked transitions: MT

Output: Prioritized test sequence: Sq

```

1: Set sequence  $Sq$  as empty,  $p = 0$ 
2: for  $k = 0 \rightarrow C_{max}$  do
3:    $B(k) = \emptyset$ 
4: end for
5: for  $t \in TS$  do
6:    $k = |C(t)|$ ;  $B(k) = B(k) \cup \{t\}$ 
7: end for
8: while  $p < |TS| - |B(0)|$  do
9:   for  $k = C_{max} \rightarrow 1$  do
10:    if  $B(k) \neq \emptyset$  then
11:      Randomly select  $t \in B(k)$  and remove it from  $B(k)$ 
12:      Append  $t$  to  $Sq$ 
13:       $p = p + 1$ 
14:    end if
15:  end for
16: end while
17: while  $B(0) \neq \emptyset$  do
18:   Randomly select  $t \in B(0)$  and remove it from  $B(0)$ 
19:   Append  $t$  to  $Sq$ 
20: end while
21: return  $Sq$ 

```

In Example 2, H8 randomly selects one test case from $B(7)$, $B(3)$, $B(2)$, and $B(1)$ by turns until all test cases from these subsets are selected in lines 8-16 of Algorithm 8. In lines 17-20, it randomly selects all test cases from $B(0)$. Subsequently, the following prioritized sequence may be created $Sq = \{t_6, t_3, t_4, t_1, t_5, t_2, t_8, t_7\}$.

D. LEARN TO RANK

In information retrieval, a typical application is to obtain a series of related documents for the user's query. The search engine sorts documents according to their relevance or importance with user's query and then delivers the sorted documents to user. However, with the increasing consideration factors, traditional prioritization model using only one sorting function becomes more sluggish. Hence, researchers try to use the machine learning algorithm to solve this rank problem.

Next, the formal description of learn to rank (LTR) is presented. Suppose there is a set of queries $Q = \{q_i\}_{i=1}^{|Q|}$. Each query q is associated with a set of documents $D_q = \{d_{q,i}\}_{i=1}^{|D_q|}$. Each query – document pair $\langle q - d_{q,j} \rangle$ is associated with a feature vector $\vec{x}_{q,j} = \vec{\psi}(q, d_{q,j})$ and a relevance label $l_{q,j}$, where ψ represents a basic ranker that can generate the real number. A ranking function f will be trained through these data. It can generate the ranking score for corresponding document. A higher score means a higher relevance or importance between the query and the document.

IV. OUR APPROACH

A. LEARN TO RANK TEST SUITE (LTR-TS)

In previous discussions, all considered heuristics focus on one single characteristic of EFSM. We attempt to propose a method that considers multiple characteristics of EFSM to rank test cases. Recently, Kim *et al.* [8] have successfully applied the LTR technique to software fault localization. Similarly, we can combine the existing heuristics to generate a new ranking model by using LTR technique. Following this idea, we present a novel MBTCP method named LTR-TS (learn to rank test suite).

Next, the formal description of LTR-TS is presented. Let $\Theta = \{d_i\}_{i=1}^{|\Theta|}$ be the set of faults, where d represents a fault. Let $TS = \{t_i\}_{i=1}^{|TS|}$ be the test suite of EFSM, where t represents one test case. Because the optimal ordering of test suite is different for different fault combinations, we can randomly choose some faults from Θ to form a set $C = \{c_i\}_{i=1}^{|C|}$, where c signifies one fault combination. When constructing the training data, each faults – test pair $\langle c - t_j \rangle$ is associated with a feature vector $\vec{x}_{c,j} = \vec{\psi}(c, t_{c,j}) = \langle \psi_1(c, t_{c,j}), \dots, \psi_{|H|}(c, t_{c,j}) \rangle \in \mathbb{R}^{|H|}$ and a label $l_{c,j}$, where ψ represents the basic heuristic that returns the position or score of test case t for corresponding fault combination c , $\mathbb{R}$ denotes the real number, and $|H|$ signifies the number of basic heuristics.

The rank function is $f : \mathbb{R}^{|H|} \rightarrow \mathbb{R}$ that assigns a score to a feature vector. Given a fault combination c , let $\vec{s}_f(c) = \langle f(\vec{x}_{c,1}), \dots, f(\vec{x}_{c,|TS|}) \rangle$ represents the vector of scores assigned by f to the test cases in TS for that c . Descending the score vector $\vec{s}_f(c)$ will get the reordered TS . The target of the LTR algorithm is to discover a function f^* in function set F , which can minimize the loss function L over the training data set D .

$$f^* = \arg \min_{f \in F} L(D; f) \quad (1)$$

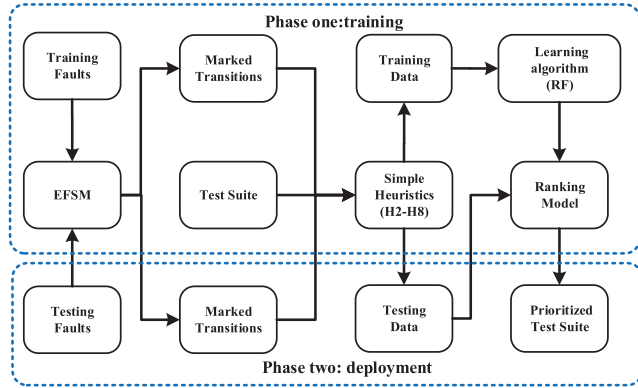


FIGURE 2. The overall process of LTR-TS.

B. THE OVERALL PROCESS OF LTR-TS

Fig. 2 shows the overall process of LTR-TS. The process consists of two major phases: training and deployment. In the training phase, faults were injected into EFSM to create different faulty versions. In our method, the mutation testing technique is used to inject faults. More detailed description of mutation testing is given in the experimental part. The transition with the injected mutation became the marked transition. Then, input the test suite and marked transition set into the simple heuristics (H2-H8) to generate different ranking results of this test suite. Next, tidied up the ranking results to complete the training data set. Finally, an LTR algorithm is utilized to build a ranking model for the considered test suite.

In the deployment phase, new faults were injected into EFSM. Same as the training phase, the test suite and the marked transition set were inputted into simple heuristics. H2 is not needed in the deployment phase because its role is to set the label of training instance in our method. Next, input the processed test data into the trained ranking model to generate the corresponding score for each test case. The test case with a higher score is prioritized for execution.

1) TRAINING DATA SET CONSTRUCTION

Fig. 3 presents an example for constructing a training data set in Monitor. Firstly, two faults d_1, d_3 are injected into the implementation of Monitor in turn to generate two faulty implementations. Then, $T3$ and $T17$ are marked based on the mapping relations between the source code and the abstract model of Monitor. Next, these two marked transitions and t_1, t_2, t_3 are imported into simple heuristics (H2-H8) to obtain the ranking results of these test cases. The numbers in the ranking results indicate the position of test cases in the reordered test suite. For instance, in the first line of ranking results, simple heuristics H2, H3, H6, and H7 put t_1 in the first position, H4 and H8 put t_1 in the second position, and H5 puts t_1 in the last position for fault combination c_1 . Similarly, inject another fault combination c_2 into Monitor will generate new ranking results for these test cases. The ranking result of H2 is used to set training labels, and the ranking results of H3-

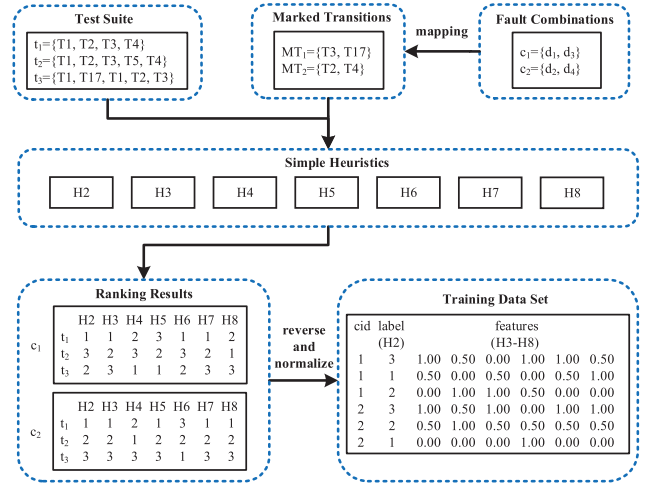


FIGURE 3. The example for constructing a training data set.

H8 are used to set feature values, but these ranking results require further processing to construct the final training data set.

The ranking model generated by LTR algorithm can be divided into the relevance ranking model and the importance ranking model [5]. In this paper, we trained a relevance ranking model. Because the larger value indicates the higher relevance between test case and fault combination in the relevance ranking model, the ranking results obtained by H2-H8 need to be reversed. In Fig. 3, the number 1 is reversed to 3, 3 is reversed to 1, while 2 does not change. Besides, the feature values are usually normalized. We use formula (2) for normalization. Here, $\min$ and $\max$ represent the minimum value and the maximum value in current feature, respectively (e.g., $\min = 1$ and $\max = 3$ in Fig. 3). After normalization, 1 will become 0.00, 2 will become 0.50, and 3 will become 1.00. In the deployment phase, the ranking results are handled in the same way.

$$x^* = \frac{x - \min}{\max - \min} \quad (2)$$

2) RF-BASED LTR ALGORITHM

The machine learning algorithm can become an LTR algorithm after doing some modifications. Random Forest (RF) is an integrated learning algorithm based on bootstrap aggregating and decision tree [34]. It can solve the problem of classification and regression. For the classification problem, entropy or gini function is used to calculate the loss, and the content in leaves are class labels. The RF-based LTR algorithm used in this paper transfers the rank problem into classification problem.

There are three pseudocodes for RF-based LTR algorithm. Algorithm 9 is the procedure for constructing a random forest with faults-level sub-sampling without replacement. D_i signifies the subset of training data set D in the i th tree. C_{D_i} implies the fault combinations in the i th tree. In line 6, the algorithm randomly selects a fault combination c that is

not contained in C_{D_i} from the total fault combinations C_D . In line 7, $\langle l_{c,j}, \vec{x}_{c,j} \rangle$ denotes one training instance in D .

Algorithm 9 Constructing the Random Forest With Faults-Level Sub-Sampling

Input: Training set D , ensemble size E , number of fault combinations per sub-sample b

Output: Tree ensemble $Trees$

```

1: function BuildForest( $D, E, b$ )
2:    $Trees \leftarrow \emptyset$ 
3:   for  $i = 1 \rightarrow E$  do
4:      $D_i \leftarrow \emptyset$ 
5:     while  $|C_{D_i}| < b$  do
6:        $c \leftarrow chooseRandom(C_D \setminus C_{D_i})$ 
7:        $D_i.add(\langle l_{c,j}, \vec{x}_{c,j} \rangle_{j=1}^{|TS|})$ 
8:     end while
9:      $Trees.add(BuildTree(D_i))$ 
10:  end for
11:  return  $Trees$ 
12: end function

```

Algorithm 10 is a pseudocode for building a decision tree. In line 9, the function *randomSubset* randomly generates a fixed-size ($\lceil \log|features| + 1 \rceil$) feature subset. For instance, there are six features in our method, so the size of feature subset is 2. In lines 10-17, the data set D_{node} in current node is sorted according to the selected feature. Then, the algorithm enumerates all possible partition points of dataset D_{node} and finds the best partition point with the maximum gain. In line 13, $Gain_{entropy}$ is a function used to calculate the gain at current partition point. It is described more in Algorithm 11. In lines 19-23, if the entropy value of the optimal partition point in current tree node is bigger than 0, the dataset D_{node} will be divided into two parts D_{left} and D_{right} according to the partition point *best.split*. Meanwhile, the corresponding children nodes are created. To improve the generalization ability of RF, we controlled the tree size by limiting the maximum number of leaves in experiments.

We utilized the expected relevance over training data instances in the particular leaf node to assign the score of test case for a given fault combination [35]. Formula (3) calculates the score of each leaf node in the training phase, where $p(l|leaf)$ denotes the relative frequency of relevance label $l \in \{1, 2, \dots, |TS|\}$ in current leaf. Hence, the ranking model that uses (3) belongs to the relevance ranking model.

$$score(leaf) = \sum_{l=1}^{|TS|} p(l|leaf) * l \quad (3)$$

In the deployment phase, formula (4) was used to calculate the average score of test case t_j for fault combination c . The average score is the final prediction result of the whole forest. Formulas (3) and (4) indicate that the RF-based LTR

algorithm has the regression characteristic.

$$f(\vec{x}_{c,j}) = \frac{1}{E} \sum_{i=1}^E score_i(leaf) \quad (4)$$

LTR algorithms can be divided into three categories: pointwise, pairwise, and listwise [5]. The pointwise approach directly calculates gain by using the feature vector of one single training instance. The pairwise approach uses the sequential relation between two different training instances to calculate gain. The listwise approach uses the whole training instances with the same query to calculate gain.

Algorithm 11 is the pseudocode for pointwise approach. The equation in this algorithm is given in [36]. Because the entropy is used, it shows the classification characteristic. In summary, the RF-based LTR algorithm used in this paper has both the classification and regression characteristics.

V. EMPIRICAL STUDY

A. RESEARCH QUESTIONS

RQ1: Whether the LTR-TS significantly outperforms random prioritization (H1)? This question will help us answer whether it is significant to introduce the LTR algorithm to MBTCP.

RQ2: How does LTR-TS compare with the H3-H8 in terms of APFD? By comparing LTR-TS with a single heuristic, we can know the effectiveness of different prioritization methods and whether LTR-TS successfully integrates H3-H8.

RQ3: How does the performance of LTR algorithms differ in terms of APFD? Answering this question will help us find out which LTR algorithm is more competitive in earlier fault detection.

RQ4: What is the time cost of different MBTCP methods? Answering this question can help us know the efficiency of different MBTCP methods.

RQ5: Is the ranking model generated by RF from a specific EFSM transferable? If the ranking model trained from a specific EFSM performs well on other EFSMs, there is no need to train a new ranking model for each EFSM. At least, this may reduce the time to train new ranking models for other EFSMs.

B. SUBJECT EFSM MODELS

We used five typical EFSMs as the experimental subjects, which are Network monitor protocol (Monitor) [10], Inres initiator protocol (INRES) [11], Class II transport protocol (Class II) [11], Link state routing protocol (OLSR) [12], and Simple connection protocol (SCP) [13]. Table 4 provides summary information of these models. Because these EFSMs do not have public test cases, an additional model-based test case generation technique is needed. There are many techniques to generate test cases for EFSM automatically (e.g., search-based algorithm [32], genetic algorithm [11]). Because the test cases generated by the search-based algorithm are all executable, we used it to generate the test case

Algorithm 10 Tree Building Process for a Random Forest**Input:** Dataset D **Output:** A Tree $root$

```

1: function BuildTree( $D$ )
2:    $root \leftarrow createNode(D)$ 
3:    $nodelist \leftarrow \{root\}$ 
4:   while  $|nodelist| > 0$  do
5:      $node \leftarrow nodelist[1]$ 
6:     Remove  $node$  from  $nodelist$ 
7:     if  $|D_{node}| > 1$  then
8:        $best \leftarrow \langle 0, null, null \rangle$ 
9:       for  $feature \in randomSubset(features, ceil(log|features| + 1))$  do
10:        for  $split \in midPoints(sort(D_{node}, feature))$  do
11:           $D_{left} \leftarrow \{\vec{x} \in D_{node} | x_{feature} < split\}$ 
12:           $D_{right} \leftarrow \{\vec{x} \in D_{node} | x_{feature} \geq split\}$ 
13:           $gain \leftarrow Gain_{entropy}(D_{node}, D_{left}, D_{right})$ 
14:          if  $gain > best.gain$  then
15:             $best \leftarrow \langle gain, feature, split \rangle$ 
16:          end if
17:        end for
18:      end for
19:      if  $best.gain > 0$  then
20:         $node.leftchild \leftarrow createNode(\vec{x} \in D_{node} | x_{best.feature} < best.split)$ 
21:         $node.rightchild \leftarrow createNode(\vec{x} \in D_{node} | x_{best.feature} \geq best.split)$ 
22:         $nodelist.add(node.leftchild, node.rightchild)$ 
23:      end if
24:    end if
25:  end while
26:  return  $root$ 
27: end function

```

Algorithm 11 Pointwise Gain**Output:** Entropy resulting from split

```

1: function  $Gain_{entropy}(D, D_{left}, D_{right})$ 
2:   return  $Entropy(D) - (\frac{|D_{left}|}{|D|} Entropy(D_{left}) + \frac{|D_{right}|}{|D|} Entropy(D_{right}))$ 
3: end function
4: Where the entropy of data  $D$  over  $L$  class labels (with  $D_l \subseteq D$  denoting examples of  $l$ th class) is computed as:

$$Entropy(D) = - \sum_{l=1}^L \frac{|D_l|}{|D|} \log \frac{|D_l|}{|D|}$$


```

for these EFSMs. In addition, we generated some random test cases to simulate the subsequent addition of test cases in the actual test. The size of test pools ranges from 462 to 1165. We selected some test cases from the test pools to form different test suites for training and testing [31].

C. MUTATION TESTING

In experiments, we applied the mutation testing technique [37] to inject faults into EFSM. Mutation testing is a defect-oriented software testing technique that uses mutation operators to simulate real software faults. After injecting a mutation, the corresponding faulty implementation (source

TABLE 4. The summary information of subject EFSMs.

EFSM name	NOS	NOT	LOC	SOP	STRP	STEP
Monitor	7	17	466	741	28	15
INRES	5	15	537	806	31	17
Class II	7	21	689	1063	35	20
OLSR	4	23	1023	1165	34	21
SCP	4	8	224	462	26	14

NOS: The number of states

NOT: The number of transitions

LOC: The line of code

SOP: The size of test pool

STRP: The size of training mutation pool

STEP: The size of testing mutation pool

code) of EFSM will be generated. At the same time, the corresponding transitions need to be marked on the EFSM specification. There are some studies [38] on the semi-automated mapping of source code and model transition, which can speed up the testing process.

There are three types of mutations used in our experiments: value mutations, decision mutations, and statement mutations [3]. Value mutations refer to the value modifications of parameters or constants (e.g., changing, adding, or removing the values). Decision mutations refer to the modifications of conditional judgment symbols (e.g., “<” change

to “>”). Statement mutations are the modifications of the statement in code (e.g., adding a new statement, deleting an old statement, and bringing forward or delaying the execution of a statement). These mutations can simulate the incorrect modifications in reality. In addition, we used *MUSIC* [39] to complete mutation testing. This tool can automatically generate many mutations, and we selected some of them for experiment. Finally, the size of training mutation pools ranges from 26 to 35, and the size of testing mutation pools ranges from 14 to 21.

D. EVALUATION METRIC

Average percentage fault detection rate (APFD) is a popular metric to evaluate the effectiveness of prioritization algorithms [40]. The value of APFD ranges from 0 to 1. If the reordered test suite has a faster fault detection rate, it will get a higher APFD value. Elbaum *et al.* [41] formally described the APFD as the following equation:

$$APFD = 1 - \frac{TF_1 + TF_2 + \dots + TF_m}{nm} + \frac{1}{2n} \quad (5)$$

In (5), T represents the test suite with n test cases, and F indicates a set of m faults revealed by T . T' represents the reordered T , and TF_i is the serial number of the first test case in T' that detects fault i [41]. Taking the optimal sequence t_2, t_3, t_1 in Example 1 to calculate APFD value would be as follows:

$$APFD = 1 - \frac{1 + 2 + 2 + 1}{3 \times 4} + \frac{1}{2 \times 3}$$

Therefore, the APFD value for test sequence t_2, t_3, t_1 should be 0.667.

E. EXPERIMENT CONFIGURATION

In total, we conducted three experiments to answer our research questions.

In Experiment 1 (**E1**), we compared H1-H8 and LTR-TS (based on RF) in different EFSMs. **E1** will answer **RQ1**, **RQ2**, and **RQ4**. In **E1**, we randomly selected 1000 mutation combinations in the training mutation pool for each EFSM in the training phase. For statistical analysis, we randomly selected 200 mutation combinations in the testing mutation pool for each EFSM in the deployment phase. Meanwhile, the execution time of each method was recorded. We used the open-source toolkit Ranklib [42] to implement RF. The parameters of RF in all experiments are as follows: the sub-sampling rate is 0.63, the bag number is 300, and the number of leaves for each tree is 200 [43].

In Experiment 2 (**E2**), we compared RF with other four LTR algorithms to answer **RQ3** and **RQ4**. The number of mutation combinations in **E2** is the same as that in **E1**. We used Ranklib to implement Multiple Additive Regression Tree (MART), RankBoost, and Coordinate Ascent (CA). In addition, we used another open-source toolkit to implement RankSVM [44]. The parameters of these four LTR algorithms are as follows: in MART, the number of trees is 1000,

and the number of leaves is 10; in RankBoost, the number of rounds to train is 200; in CA, the number of random restarts is 2, the number of iterations to search in each dimension is 25, and the performance tolerance between two solutions is 0.001; in RankSVM, the c value is 20. These parameters are defaulted by the tools.

In Experiment 3 (**E3**), the ranking model of Monitor trained by RF was transferred to other EFSMs for ranking. **E3** will answer **RQ5**. The transfer ranking model was compared with the new ranking model trained by RF in the other four EFSMs.

Our experimental environment is as follows: CentOS, Intel Xeon E5-2450 CPU, 32G RAM.

F. RESULT AND DISCUSSION

1) WHETHER THE LTR-TS SIGNIFICANTLY OUTPERFORMS RANDOM PRIORITIZATION (H1)?

In this section, we compared LTR-TS (based on RF) with H1 in terms of APFD. We drew the box-and-whisker plots of the APFD values for H1-H8 and LTR-TS in five EFSMs. In Fig. 4, the x -axis represents different MBTCP methods, and the y -axis represents APFD values. Fig. 4 (a) - Fig. 4(e) show the box plots of APFD values for each method in different EFSMs. Fig. 4 (f) shows the box plots of cumulative APFD values for each method in five EFSMs. As can be intuitively observed from Fig. 4, RF gets much higher APFD values than H1 in every EFSM. Table 5 provides the statistic data of APFD for H1-H8 and RF. The statistics include minimum, maximum, mean, and standard deviation (Std-Dev). In total, the minimum APFD value of RF is 0.717, which is much higher than that of H1 (0.314). The maximum APFD value and the mean APFD value of RF are 2.4% and 33.9% higher than those of H1, respectively. The standard deviation of RF is 0.041, which is much lower than that of H1 (0.115). These comparisons indicate that RF improves greatly in both effectiveness and stability compared with H1 in terms of APFD. The reason may be that H1 does not utilize any information from EFSM for ranking.

In conclusion, LTR-TS is better than H1 in terms of APFD, and it is feasible to introduce the LTR technique to MBTCP.

2) HOW DOES LTR-TS COMPARE WITH THE H3-H8 IN TERMS OF APFD?

In this section, we compared LTR-TS (based on RF) with the existing heuristics (H3-H8) in terms of APFD. From Fig. 4, we can clearly observe that RF has the best performance in every EFSM. H4 and H6 have similar performance and are better than H3, H5, H7, and H8. H6 performs better than H4 except in SCP. In general, H4 and H6 are two heuristics with strong competitive performance.

We conducted a one-way analysis of variance (ANOVA) [3] to determine whether there are significant differences of APFD values between different MBTCP methods. The null hypothesis is that the APFD values of different MBTCP methods are equal (in total). We set the significance level

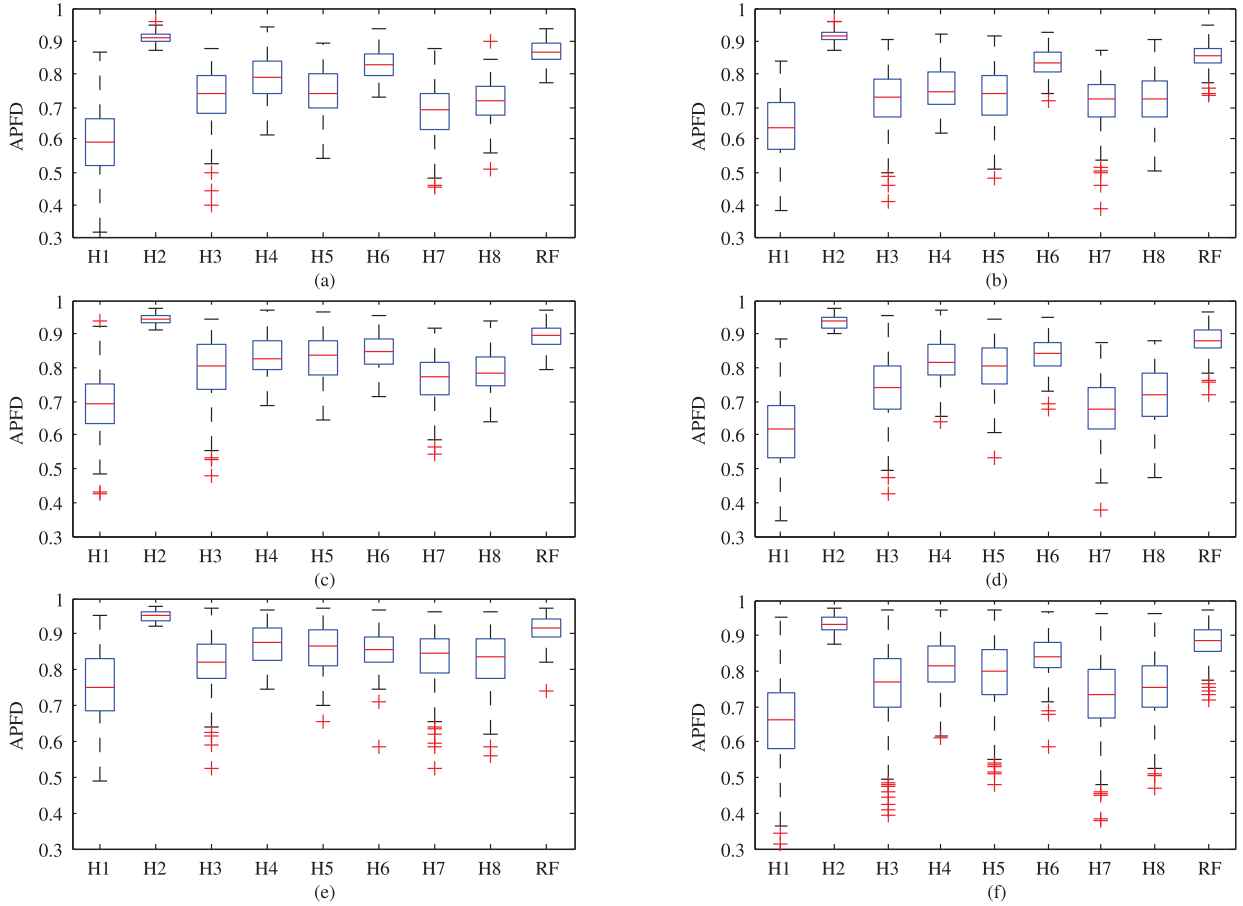


FIGURE 4. The box plots of APFD values for MBTCP methods in different EFSMs. (a) Monitor; (b) INRES; (c) Class II; (d) OLSR; (e) SCP; (f) All EFSMs.

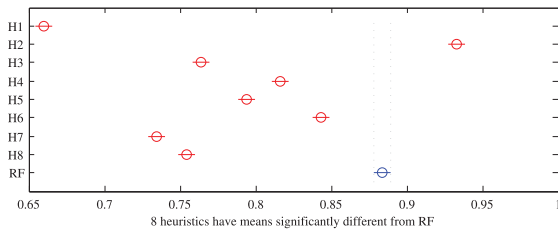


FIGURE 5. Results of hypothesis testing for cumulative data for all EFSMs.

to 5%. The p value returned by the experiment is much less than 0.05, which rejects the null hypothesis and indicates there are significant differences between these MBTCP methods.

Furthermore, we utilized the multiple comparison procedure to investigate whether there are significant differences of the mean APFD value between one method and other methods at the level of 5%. Fig. 5 shows the hypothesis testing results. The blue line represents the result of RF, and the red line represents the result of the method that has a significant difference with RF. We can observe that RF is significantly different from other methods. We drew Table 6 to compare these methods more intuitively. The symbol ($>$, $=$, $<$) indicates whether the method in a row is better,

with no significant difference, or worse than the method in a column, respectively. The method in a row with more “ $>$ ” indicates it has a better performance in faster faults detection. We can see that RF has the most “ $>$ ” except for the optimal algorithm H2.

From Table 5, we can obtain that the minimum APFD value of RF is 17.2% higher than that of H4 and 90.2% higher than that of H7 in total. The maximum APFD value of RF is slightly higher than that of H3-H8 in total. The mean APFD value of RF is 4.9% higher than that of H6 and 20.4% higher than that of H7 in total. The standard deviation of RF is 14.6% lower than that of H6 and 59% lower than that of H7 in total. The reason for the above comparison results may be that H3-H8 only consider one aspect of EFSM, while LTR-TS successfully considers the multiple characteristics of EFSM by integrating H3-H8.

As a summary, we can conclude that LTR-TS has different degrees of improvement in effectiveness and stability compare with H3-H8.

3) HOW DOES THE PERFORMANCE OF LTR ALGORITHMS DIFFER IN TERMS OF APFD?

This section compared MART, RankBoost, CA, RankSVM, and RF in terms of APFD. Fig. 6 (a) - Fig. 6 (e) are the box

TABLE 5. The statistic data of APFD for H1-H8 and RF in different EFSMs.

EFSMs	Comparisons	Prioritization Methods								
		H1	H2	H3	H4	H5	H6	H7	H8	RF
Monitor	Min	0.314	0.875	0.394	0.612	0.539	0.733	0.451	0.510	0.774
	Max	0.868	0.966	0.878	0.944	0.896	0.941	0.878	0.901	0.942
	Mean	0.597	0.916	0.731	0.794	0.745	0.835	0.684	0.721	0.872
	Std-Dev	0.106	0.017	0.091	0.066	0.073	0.045	0.082	0.064	0.035
INRES	Min	0.379	0.875	0.409	0.617	0.479	0.718	0.384	0.504	0.735
	Max	0.842	0.966	0.910	0.925	0.920	0.933	0.875	0.909	0.950
	Mean	0.639	0.919	0.726	0.758	0.735	0.838	0.714	0.722	0.859
	Std-Dev	0.097	0.018	0.093	0.067	0.086	0.041	0.081	0.076	0.038
Class II	Min	0.424	0.911	0.479	0.685	0.645	0.713	0.541	0.640	0.797
	Max	0.940	0.977	0.942	0.970	0.963	0.955	0.918	0.939	0.971
	Mean	0.696	0.943	0.797	0.837	0.829	0.846	0.766	0.785	0.894
	Std-Dev	0.092	0.015	0.092	0.058	0.069	0.049	0.073	0.062	0.035
OLSR	Min	0.343	0.899	0.426	0.639	0.532	0.677	0.377	0.471	0.717
	Max	0.886	0.973	0.955	0.968	0.946	0.949	0.876	0.881	0.966
	Mean	0.616	0.936	0.741	0.818	0.802	0.841	0.677	0.717	0.881
	Std-Dev	0.107	0.018	0.100	0.064	0.073	0.051	0.093	0.080	0.041
SCP	Min	0.491	0.922	0.523	0.747	0.653	0.584	0.526	0.559	0.743
	Max	0.952	0.976	0.974	0.968	0.973	0.967	0.961	0.964	0.975
	Mean	0.753	0.950	0.824	0.874	0.862	0.856	0.831	0.827	0.915
	Std-Dev	0.095	0.014	0.078	0.055	0.066	0.052	0.079	0.076	0.032
Total	Min	0.314	0.875	0.394	0.612	0.479	0.584	0.377	0.471	0.717
	Max	0.952	0.977	0.974	0.970	0.973	0.967	0.961	0.964	0.975
	Mean	0.660	0.933	0.764	0.816	0.794	0.843	0.734	0.754	0.884
	Std-Dev	0.115	0.021	0.099	0.073	0.088	0.048	0.100	0.084	0.041

TABLE 6. Result of hypothesis testing for cumulative data for all EFSMs.

	H1	H2	H3	H4	H5	H6	H7	H8	RF
H1	—	<	<	<	<	<	<	<	<
H2	>	—	>	>	>	>	>	>	>
H3	>	<	—	<	<	<	<	=	<
H4	>	<	>	—	>	<	>	>	<
H5	>	<	>	<	—	<	>	>	<
H6	>	<	>	>	>	—	>	>	<
H7	>	<	>	<	<	<	—	<	<
H8	>	<	=	<	<	<	>	—	<
RF	>	<	>	>	>	>	>	>	—

plots of APFD values for each LTR algorithm in different EFSMs. Fig. 6 (f) shows the box plots of cumulative APFD values for each LTR algorithm in five EFSMs. The x -axis represents different LTR algorithms, and the y -axis represents APFD values. We can observe that MART, RankSVM, and RF perform better than RankBoost and CA.

We also conducted the ANOVA analysis to find out whether there are significant differences of the APFD values between different LTR algorithms. The null hypothesis is that the APFD values of different LTR algorithms are equal (in total). We set the significance level to 5%. The experiment result of p is much less than 0.05, which rejects the

TABLE 7. Result of hypothesis testing for cumulative data for all EFSMs.

	MART	RankBoost	CA	RankSVM	RF
MART	—	>	>	=	<
RankBoost	<	—	>	<	<
CA	<	<	—	<	<
RankSVM	=	>	>	—	<
RF	>	>	>	>	—

null hypothesis and indicates there are significant differences between these LTR algorithms.

Moreover, we made multiple comparisons about the mean APFD value differences between one LTR algorithm and other LTR algorithms at the level of 5%. Fig. 7 and Table 7 present the hypothesis testing results. The hypothetical result of MART is gray, which suggests there is no significant difference of the mean APFD value between MART and RankSVM. However, from Fig. 6, we can observe that RankSVM is more stable than MART. Table 7 shows that RF has the best performance, followed by RankSVM and MART, then is RankBoost, and that CA performs the worst. From Table 8, we can find that the mean APFD value of RF is the highest among the five LTR algorithms in total, while it is the opposite in terms of standard deviation.

We think that RF and MART perform well might be because their ranking models are tree models. The benefit

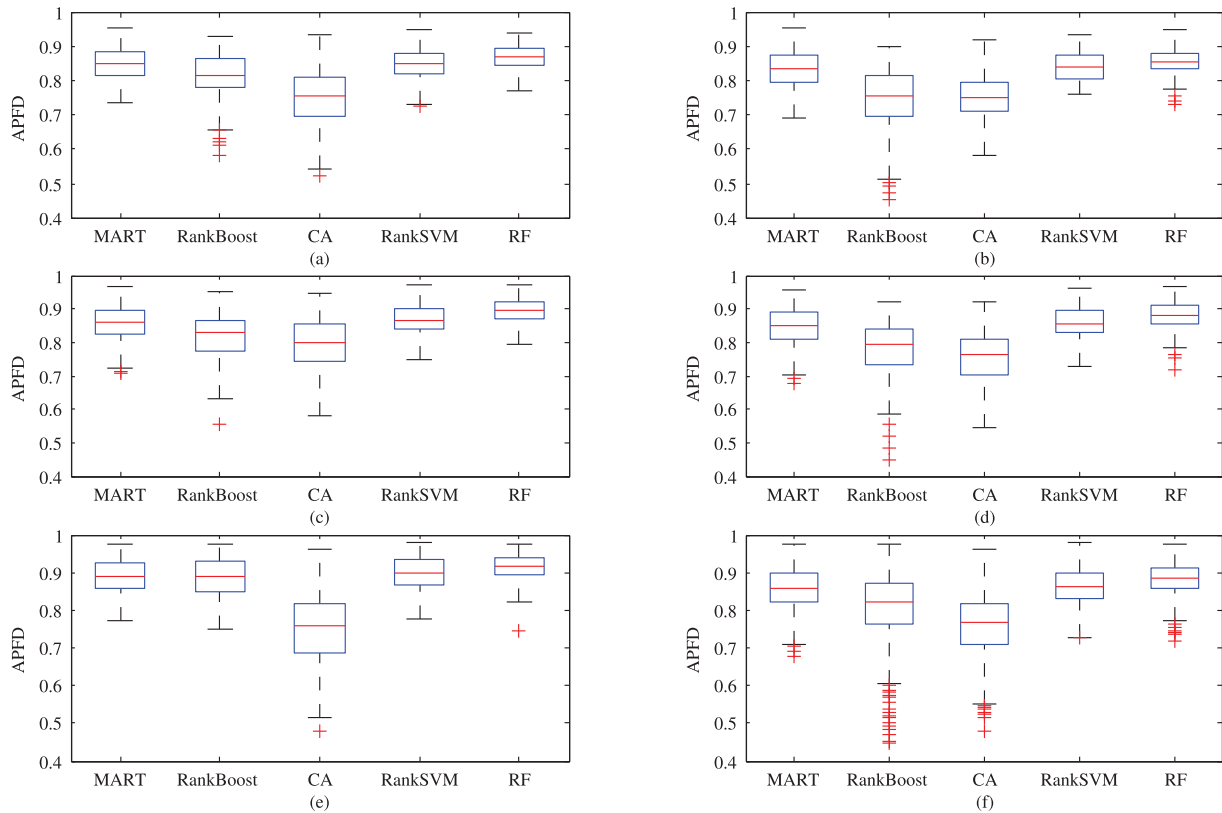


FIGURE 6. The box plots of APFD values for LTR algorithms in different EFSMs. (a) Monitor; (b) INRES; (c) Class II; (d) OLSR; (e) SCP; (f) All EFSMs.

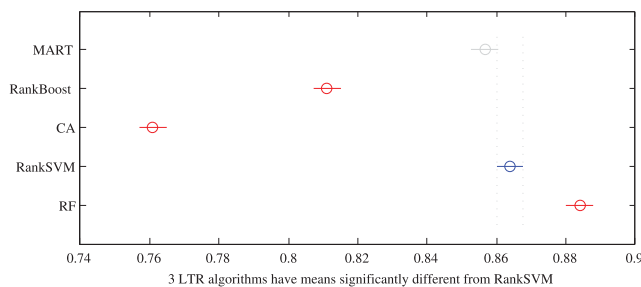


FIGURE 7. Result of hypothesis testing for cumulative data for all EFSMs.

of the tree model is that it can retain more decision-making information. RF is better than MART because it increases the diversity of decision tree by randomly selecting samples and features. RankSVM uses the kernel function to map the problem to a high-dimensional space, so its performance is not bad. Finally, CA's implementation is relatively simple, so its performance is the worst in the five LTR algorithms.

Eventually, we can conclude that RF is more effective than other four LTR algorithms in terms of APFD.

4) WHAT IS THE TIME COST OF DIFFERENT MBTCP METHODS?

Table 9 presents the average execution time (in seconds) for different methods in five EFSMs. 0.01 seconds is the minimum record time in our experiment. In total, we can observe from the table that H1 requires the least execution

time of 0.26 seconds, and H6 requires the most execution time of 1.20 seconds in the simple heuristics. The reason may be that H1 does not use any information from EFSM, while H6 is relatively complex and needs to read the information of marked transitions. In the LTR algorithms, MART and RF are the most time-consuming. The possible reason is that the ranking model generated by MART and RF are comparatively larger (tree model). In addition, we can find that all methods require the least time in SCP. This may because that SCP has the fewest test cases and the simplest model structure in these EFSMs.

In conclusion, the LTR algorithm is more time-consuming than the simple heuristic because it needs extra time to read feature value and calculate score. Moreover, it takes much time to construct the training data set. But this is inevitable for the supervised machine learning algorithm. Despite this, the LTR algorithm improves significantly compared with simple heuristics in terms of APFD.

5) IS THE RANKING MODEL GENERATED BY RF FROM A SPECIFIC EFSM TRANSFERABLE?

In this section, we discussed the transferability of the ranking model trained by RF. In Fig. 8, the blue box represents the APFD values for transfer ranking model, and the red box represents the APFD values for new ranking model. As can be seen from the figure, the ranking model of Monitor can be applied in other EFSMs.

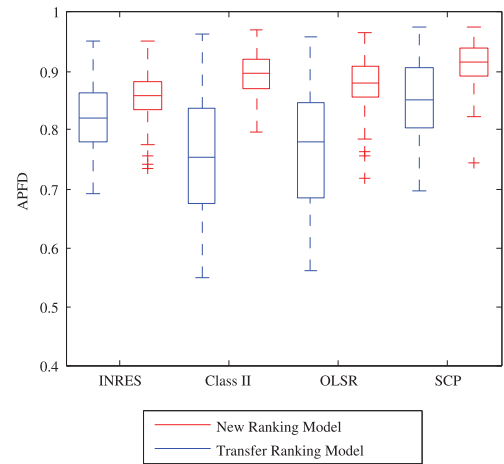
TABLE 8. The statistic data of APFD for LTR algorithms in different EFSMs.

EFSMs	Comparisons	LTR algorithms				
		MART	RankBoost	CA	RankSVM	RF
Monitor	Min	0.736	0.581	0.523	0.725	0.774
	Max	0.956	0.934	0.938	0.953	0.942
	Mean	0.851	0.817	0.754	0.852	0.872
	Std-Dev	0.046	0.061	0.077	0.044	0.035
INRES	Min	0.692	0.452	0.583	0.762	0.735
	Max	0.957	0.900	0.924	0.939	0.950
	Mean	0.837	0.746	0.755	0.841	0.859
	Std-Dev	0.051	0.089	0.061	0.043	0.038
Class II	Min	0.709	0.554	0.580	0.748	0.797
	Max	0.969	0.949	0.945	0.973	0.971
	Mean	0.857	0.821	0.792	0.867	0.894
	Std-Dev	0.053	0.065	0.074	0.046	0.035
OLSR	Min	0.678	0.447	0.546	0.726	0.717
	Max	0.956	0.922	0.923	0.964	0.966
	Mean	0.848	0.783	0.754	0.859	0.881
	Std-Dev	0.058	0.085	0.074	0.047	0.041
SCP	Min	0.774	0.748	0.477	0.779	0.743
	Max	0.975	0.975	0.962	0.979	0.975
	Mean	0.890	0.889	0.750	0.901	0.915
	Std-Dev	0.043	0.047	0.093	0.042	0.032
Total	Min	0.678	0.447	0.477	0.725	0.717
	Max	0.975	0.975	0.962	0.979	0.975
	Mean	0.857	0.811	0.761	0.864	0.884
	Std-Dev	0.054	0.086	0.078	0.049	0.041

TABLE 9. The average time cost of different MBTCP methods.

Methods	Monitor	EFSMs				
		INRES	Class II	OLSR	SCP	Total
H1	0.21	0.19	0.41	0.40	0.11	0.26
H2	0.34	0.29	0.58	0.69	0.22	0.42
H3	0.37	0.33	0.63	0.75	0.25	0.47
H4	0.66	0.64	1.07	0.99	0.43	0.76
H5	0.81	0.72	1.25	1.13	0.53	0.89
H6	1.01	1.08	1.67	1.57	0.66	1.20
H7	0.57	0.67	1.04	1.05	0.36	0.74
H8	0.66	0.77	1.05	1.11	0.52	0.82
MART	2.47	2.59	3.96	3.75	2.06	2.97
RankBoost	1.83	1.98	3.19	3.13	1.81	2.39
CA	1.68	1.57	3.02	2.98	1.99	2.25
RankSVM	2.09	2.22	3.45	3.32	1.48	2.51
RF	2.77	2.89	4.33	4.05	2.03	3.21

Table 10 provides detailed transfer learning data. The mean APFD value of transfer ranking model is lower than that of new ranking model in every EFSM, while it is the opposite in terms of the standard deviation, which reveals that the transfer ranking model is not as good as the new ranking model. Besides, the transfer ranking model performs worse in Class II and OLSR than in INRES and SCP. This may be related to the difference in the size of the test suite and the structure of EFSM. The ranking model of Monitor performs

**FIGURE 8.** The box plots of APFD values for transfer learning.**TABLE 10.** The statistic data of APFD for transfer learning.

Ranking Model	Comparasions	EFSMs			
		INRES	Class II	OLSR	SCP
New Ranking Model	Min	0.735	0.797	0.717	0.743
	Max	0.950	0.971	0.966	0.975
	Mean	0.859	0.894	0.881	0.915
	Std-Dev	0.038	0.035	0.041	0.032
Transfer Ranking Model	Min	0.692	0.549	0.560	0.696
	Max	0.951	0.962	0.959	0.975
	Mean	0.822	0.756	0.770	0.853
	Std-Dev	0.058	0.098	0.097	0.063

poorly in these two EFSMs because its model structure and test suite size are relatively different from those of Class II and OLSR.

In the end, we can conclude that the ranking model trained by RF has a certain transferability, which may be related to the structure of EFSM and the size of training test suite.

VI. THREATS TO VALIDITY

The first threat to internal validation is the use of open-source toolkits in experiments. To ensure the accuracy of use, we carefully checked the process of the experiment. The parameters are defaulted by the tools or given in the reference. Secondly, the specification of EFSM may be designed with errors in reality. In this paper, the EFSMs are derived from the literature and have sufficient information to be implemented. Also, the implementation of EFSM may have errors, which will result in generating the incorrect test suite. To avoid this risk, we validated the implementation version of the EFSM without the seeded mutation. Moreover, although the markup process from source code modification to model transition is simple, there is still a risk of markup errors. Therefore, we added a validation process to experiments to ensure that the marked transitions are correctly marked.

For external validation, first of all, the seeded mutations may not represent real faults. However, some literature

has shown that these mutation operators can replace real faults [3]. In fact, we seeded different mutation operators to reduce the influence of this threat in experiments. Another threat to external validation is that the experimental EFSMs are all communication protocol models. However, there are significant differences in the structural complexity and the number of states and transitions of these EFSMs. Besides, there is no direct correlation between the EFSM model and the application domain. Therefore, we believe that the experimental EFSMs are sufficiently representative. In the future, we will use more EFSMs for experiments.

Finally, the APFD metric may affect the construct validation of the experiment because it does not consider the time cost of test case execution and the fault security level. For this problem, we will use $APFD_C$ as a metric to evaluate the ranking results in the future.

VII. CONCLUSION

In this paper, we introduced the learn-to-rank technique to MBTCP. Firstly, we described several existing MBTCP techniques, which are presented by other researchers. Then, we described our method in detail. The effectiveness and efficiency of our method and existing methods are compared through detailed experiments. The experimental results show that our method outperforms the existing methods in terms of APFD. Meanwhile, different learn-to-rank algorithms are compared. It can be concluded that the RF-based LTR algorithm is more effective in MBTCP. In addition, we find that the ranking model generated by the RF-based LTR algorithm has a certain transferability.

In the future, more complex EFSMs with larger test suites will be considered. Furthermore, we will try to apply more features in our method. The influence of different parameters of the LTR algorithms will also be observed.

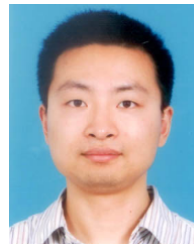
REFERENCES

- [1] S. Yoo and M. Harman, "Regression testing minimization, selection and prioritization: A survey," *Softw. Test., Verification Rel.*, vol. 22, no. 2, pp. 67–120, Mar. 2012.
- [2] S. Elbaum, A. G. Malishevsky, and G. Rothermel, "Prioritizing test cases for regression testing," in *Proc. ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, 2000, pp. 102–112.
- [3] L. Tahat, B. Korel, G. Koutsogiannakis, and N. Almasri, "State-based models in regression test suite prioritization," *Softw. Qual. J.*, vol. 25, no. 3, pp. 703–742, Sep. 2017.
- [4] B. Korel and G. Koutsogiannakis, "Experimental comparison of code-based and model-based test prioritization," in *Proc. Int. Conf. Softw. Test., Verification, Validation Workshops*, 2009, pp. 77–84.
- [5] T.-Y. Liu, "Learning to rank for information retrieval," *Found. Trends Inf. Retr.*, vol. 3, no. 3, pp. 225–331, 2011.
- [6] H. Li, "A short introduction to learning to rank," *IEICE Trans. Inf. Syst.*, vol. E94-D, no. 10, pp. 1854–1862, 2011.
- [7] T.-D. B. Le, D. Lo, C. Le Goues, and L. Grunske, "A learning-to-rank based fault localization approach using likely invariants," in *Proc. 25th Int. Symp. Softw. Test. Anal.*, Jul. 2016, pp. 177–188.
- [8] Y. Kim, S. Mun, S. Yoo, and M. Kim, "Precise learn-to-rank fault localization using dynamic and static features of target programs," *ACM Trans. Softw. Eng. Methodol.*, vol. 28, no. 4, pp. 1–34, Oct. 2019.
- [9] D. Zou, J. Liang, Y. Xiong, M. D. Ernst, and L. Zhang, "An empirical study of fault localization families and their combinations," *IEEE Trans. Softw. Eng.*, early access, Jan. 10, 2019, doi: [10.1109/TSE.2019.2892102](https://doi.org/10.1109/TSE.2019.2892102).
- [10] C. Huang, M. Jang, and Y. Lin, "Executable EFSM-based data flow and control flow protocol test sequence generation using reachability analysis," *J. Chin. Inst. Eng.*, vol. 22, no. 5, pp. 593–615, Jul. 1999.
- [11] A. S. Kalaji, R. M. Hierons, and S. Swift, "An integrated search-based approach for automatic testing from extended finite state machine (EFSM) models," *Inf. Softw. Technol.*, vol. 53, no. 12, pp. 1297–1318, Dec. 2011.
- [12] J.-M. Orset, B. Alcalde, and A. Cavalli, "An EFSM-based intrusion detection system for ad hoc networks," in *Proc. Int. Symp. Automated Technol. Verification Anal.* Berlin, Germany: Springer, 2005, pp. 400–413.
- [13] A. Cavalli, C. Gervy, and S. Prokopenko, "New approaches for passive testing using an extended finite state machine specification," *Inf. Softw. Technol.*, vol. 45, no. 12, pp. 837–852, Sep. 2003.
- [14] S. Yoo, M. Harman, P. Tonella, and A. Susi, "Clustering test cases to achieve effective and scalable prioritization incorporating expert knowledge," in *Proc. 18th Int. Symp. Softw. Test. Anal.*, 2009, pp. 201–212.
- [15] A. Marchetto, M. M. Islam, W. Asghar, A. Susi, and G. Scanniello, "A multi-objective technique to prioritize test cases," *IEEE Trans. Softw. Eng.*, vol. 42, no. 10, pp. 918–940, Oct. 2016.
- [16] D. Di Nucci, A. Panichella, A. Zaidman, and A. De Lucia, "A test case prioritization genetic algorithm guided by the hypervolume indicator," *IEEE Trans. Softw. Eng.*, vol. 46, no. 6, pp. 674–696, Jun. 2020.
- [17] Q. Luo, K. Moran, L. Zhang, and D. Poshvanyk, "How do static and dynamic test case prioritization techniques perform on modern software systems? An extensive study on GitHub projects," *IEEE Trans. Softw. Eng.*, vol. 45, no. 11, pp. 1054–1080, Nov. 2019.
- [18] M. Laali, H. Liu, M. Hamilton, M. Spichkova, and H. W. Schmidt, "Test case prioritization using online fault detection information," in *Proc. Ada-Eur. Int. Conf. Reliable Softw. Technol.* Cham, Switzerland: Springer, 2016, pp. 78–93.
- [19] M. Khatibsyarhini, M. A. Isa, D. N. A. Jawawi, H. N. A. Hamed, and M. D. M. Suffian, "Test case prioritization using firefly algorithm for software testing," *IEEE Access*, vol. 7, pp. 132360–132373, 2019.
- [20] H. Jahan, Z. Feng, and S. M. H. Mahmud, "Risk-based test case prioritization by correlating system methods and their associated risks," *Arabian J. Sci. Eng.*, vol. 45, pp. 1–14, Apr. 2020.
- [21] K. Solanki, Y. Singh, S. Dalal, and P. R. Srivastava, "Test case prioritization: An approach based on modified ant colony optimization," in *Emerging Research in Computing, Information, Communication and Applications*. 2016.
- [22] D. Kundu, M. Sarma, D. Samanta, and R. Mall, "System testing for object-oriented systems with test case prioritization," *Softw. Test., Verification Rel.*, vol. 19, no. 4, pp. 297–333, Dec. 2009.
- [23] P. Kaur, P. Bansal, and R. Sibal, "Prioritization of test scenarios derived from UML activity diagram using path complexity," in *Proc. CUBE Int. Inf. Technol. Conf.*, 2012, pp. 355–359.
- [24] J. F. S. Ouriques, E. G. Cartaxo, and P. D. L. Machado, "Test case prioritization techniques for model-based testing: A replicated study," *Softw. Qual. J.*, vol. 26, no. 4, pp. 1451–1482, Dec. 2018.
- [25] S. Sharma and A. Singh, "Model-based test case prioritization using ACO: A review," in *Proc. 4th Int. Conf. Parallel, Distrib. Grid Comput. (PDGC)*, 2016, pp. 177–181.
- [26] M. L. Mohd-Shafie, W. M. N. Wan-Kadir, M. Khatibsyarhini, and M. A. Isa, "Model-based test case prioritization using selective and even-spread count-based methods with scrutinized ordering criterion," *PLoS ONE*, vol. 15, no. 2, Feb. 2020, Art. no. e0229312.
- [27] Y. Hu, Q. Da, A. Zeng, Y. Yu, and Y. Xu, "Reinforcement learning to rank in E-commerce search engine: Formalization, analysis, and application," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 368–377.
- [28] H. Niu, I. Keivanloo, and Y. Zou, "Learning to rank code examples for code search engines," *Empirical Softw. Eng.*, vol. 22, no. 1, pp. 259–291, Feb. 2017.
- [29] X. Ye, R. Bunescu, and C. Liu, "Learning to rank relevant files for bug reports using domain knowledge," in *Proc. ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, 2014, pp. 689–699.
- [30] X. Yang, K. Tang, and X. Yao, "A learning-to-rank approach to software defect prediction," *IEEE Trans. Rel.*, vol. 64, no. 1, pp. 234–246, Mar. 2015.
- [31] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Prioritizing test cases for regression testing," *IEEE Trans. Softw. Eng.*, vol. 27, no. 10, pp. 929–948, Oct. 2001.
- [32] T. Shu, Z. Ding, M. Chen, and J. Xia, "A heuristic transition executability analysis method for generating EFSM-specified protocol test sequences," *Inf. Sci.*, vols. 370–371, pp. 63–78, Nov. 2016.

- [33] K. El-Fakih, A. Simao, N. Jadoon, and J. Carlos Maldonado, "On studying the effectiveness of extended finite state machine based test selection criteria," in *Proc. IEEE 7th Int. Conf. Softw. Test., Verification Validation Workshops*, Apr. 2014, pp. 222–229.
- [34] L. Guo, Y. Ma, B. Cukic, and H. Singh, "Robust prediction of fault-proneness by random forests," in *Proc. 15th Int. Symp. Softw. Rel. Eng.*, 2004, pp. 417–428.
- [35] M. Ibrahim, "An empirical comparison of random forest-based and other learning-to-rank algorithms," *Pattern Anal. Appl.*, vol. 23, pp. 1–23, Oct. 2019.
- [36] A. Criminisi, "Decision forests: A unified framework for classification, regression, density estimation, manifold learning and semi-supervised learning," *Found. Trends Comput. Graph. Vis.*, vol. 7, nos. 2–3, pp. 81–227, 2011.
- [37] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: An analysis and survey," in *Advances in Computers*, vol. 112. Amsterdam, The Netherlands: Elsevier, 2019, pp. 275–378.
- [38] A. Paiva, J. Faria, N. Tillmann, and R. Vidal, "A model-to-implementation mapping tool for automated model-based gui testing," in *Proc. 7th Int. Conf. Formal Methods Softw. Eng.*, 2005, pp. 450–464.
- [39] D. L. Phan, Y. Kim, and M. Kim, "MUSIC: Mutation analysis tool with high configurability and extensibility," in *Proc. IEEE Int. Conf. Softw. Test., Verification Validation Workshops (ICSTW)*, Apr. 2018, pp. 40–46.
- [40] Y. T. Yu and M. F. Lau, "Fault-based test suite prioritization for specification-based testing," *Inf. Softw. Technol.*, vol. 54, no. 2, pp. 179–202, Feb. 2012.
- [41] S. Elbaum, A. G. Malishevsky, and G. Rothermel, "Test case prioritization: A family of empirical studies," *IEEE Trans. Softw. Eng.*, vol. 28, no. 2, pp. 159–182, Aug. 2002.
- [42] V. Dang. (2013). *The Lemur Project-Wiki-Ranklib*. [Online]. Available: <http://sourceforge.net/p/lemur/wiki/RankLib>
- [43] M. Ibrahim and M. Carman, "Comparing pointwise and listwise objective functions for random-forest-based learning-to-rank," *ACM Trans. Inf. Syst.*, vol. 34, no. 4, pp. 1–38, Sep. 2016.
- [44] T. Joachims, "Optimizing search engines using clickthrough data," in *Proc. 8th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2002, pp. 133–142.



YECHAO HUANG was born in February 1996. He received the B.S. degree in electronic and information engineering from Taizhou University, in 2018. He is currently pursuing the M.S. degree in computer science with Zhejiang Sci-Tech University. His current research interests include model-based testing and test case prioritization.



TING SHU was born in July 1979. He received the Ph.D. degree in computer science from Zhejiang University, in 2010. He is currently an Associate Professor with the School of Information Science and Technology, Zhejiang Sci-Tech University, Hangzhou, China. His current research interests include software testing and network protocol testing.



ZUOHUA DING (Member, IEEE) received the M.S. degree in computer science and the Ph.D. degree in mathematics from the University of South Florida, Tampa, FL, USA, in 1996 and 1998, respectively. He is currently a Professor and the Director of the Laboratory of Intelligent Computing and Software Engineering, Zhejiang Sci-Tech University, Hangzhou, China. He has authored or coauthored more than 70 articles. His current research interests include system modeling, software reliability prediction, intelligent software systems, and service robots.

• • •